חוברת היסוד לספר הקורס המאוחד

פרק 1 — היכרות חוזרת עם Python למפתח AI

מהקוד הראשון ועד לעיבוד נתונים, פונקציות ומערכים מספריים

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

ספר תלמיד עצמאי • הסברים מלאים • דוגמאות מקוריות • קוד מודרך • תרגול עצמי



תהליך הלמידה בספר: לומדים לכתוב קוד כדי לשאול שאלות על נתונים — ורק בהמשך לאמן מודלים.

שאלה פותחת איך נכתוב תוכנית שמקבלת מדידות של מחשבים, מזהה מצב חריג ומכינה נתונים למחקר — בלי לאבד את היכולת להסביר כל שורה?

בפרק הזה דודו אחראי על מעבדת מחשבים בבית הספר. הוא אוסף טמפרטורות, שעות שימוש ונתוני תקלות. בתחילה הוא מחשב ממוצע ומסמן מדידות גבוהות. בהמשך הוא מעביר את הפעולות לפונקציות ומייצג את הנתונים במערכים. כל פעולה נועדה לפתור בעיה אמיתית, לא רק להדגים תחביר.

הפרק מבוסס על מפת תכנית ההתמחות שבמאגר הקורס שלנו: פרק 1 — היכרות חוזרת עם שפת תכנות, 30 שעות. חלוקת השעות הפנימית ומקרי החקר הם תכנון פדגוגי מקורי; אין בכך טענה שהפרטים מופיעים מילה במילה בסילבוס הרשמי.

מפת הדרך של הפרק

יחידת לימוד	מוקד	שעות מוצעות
1.1	סביבת Google Colab, קוד ונתונים	3
1.2	משתנים, טיפוסים, פעולות וחישובים	3
1.3	מחרוזות, קלט והדפסה קריאה	3
1.4	תנאים, ביטויים בוליאניים ולוגיקה	3
1.5	לולאות, סכימה וספירה	3
1.6	רשימות, אינדקסים והעתקת נתונים	3
1.7	מילונים ומבנה של רשומה	3
1.8	פונקציות, תכנון קוד ובדיקות	3
1.9	שגיאות, מודולים וקובצי נתונים	3
1.10	NumPy ופרויקט מסכם	3

בכל יחידה: בעיה קצרה, הסבר מושגי, דוגמה פתורה, קוד, טעויות נפוצות, משימות העברה וכרטיס יציאה. חלק מהמיומנויות מוכרות מיחידה 3; כאן מעמיקים בהן כדי להכין קוד יציב וקריא לפרויקט למידה עמוקה.

מפת הקשרים בין שלוש תכניות הלימוד

לאן ממשיכים	מה כבר נלמד / מה נלמד כאן	מסלול
הפעלת פונקציות וקוד עצמאי שאפשר להסביר בעל־פה	היכרות עם Colab, משתנים, רשימות, Pandas וניתוח ראשוני	יחידה 3 — כיתה י׳
הבנה מתמטית של פעולות NumPy ומרחקים	וקטורים, מטריצת מאפיינים, תיאור נתונים ואלגוריתמים	יחידה 5 — נושאי ליבה
OOP, EDA, טנזורים ומערכת פרויקט גמר	השלמת יסודות Python, קבצים, פונקציות ובדיקות	למידה עמוקה — פרק 1

מפת הקשרים היא תכנון דידקטי המתבסס על המסלולים במאגר; אין לראות בכל שורה דרישת חובה זהה בשלוש תכניות הלימודים.

מה נדרש לדעת לפני שמתחילים?

נדרשת היכרות בסיסית עם מושגי קלט ופלט, משתנה ופעולת חשבון. מי שכבר למד Python יוכל לבצע את מבדק הפתיחה ולעבוד מהר יותר בחלקי החזרה, אבל חשוב שלא לדלג על תרגול של פונקציות, רשימות ומבני נתונים.

מבדק פתיחה — ללא מחשב

1. מה ההבדל בין המספר 7 לבין המחרוזת "7"? איזו פעולה אפשר לבצע על כל אחד מהם?
2. אם ברשימה יש חמישה איברים, מהו האינדקס של האיבר האחרון?
3. מה פירוש הביטוי "הפונקציה מחזירה ערך"? במה הוא שונה מהדפסת ערך?
4. דודו קיבל 4 מדידות: 70, 82, 75, 89. כיצד יחשב ממוצע ומספר מדידות של 80 ומעלה?

מילון סימנים מוקדם

סימן	משמעות
=	השמה של ערך למשתנה
==	השוואת שני ערכים
:	תחילת בלוק כמו if, for או def
[]	רשימה, אינדקס או חיתוך
()	קריאה לפונקציה או קיבוץ ביטוי
None	היעדר ערך שימושי, ולא המספר אפס
shape	ממדי מערך NumPy; לא מופיע ברשימת Python רגילה

1.1 — סביבת עבודה, תאים וסדר הרצה

1 עובדים במחברת המורכבת מתאים. תא טקסט מאפשר לתאר את שאלת המחקר; תא Google Colab קוד מריץ הוראות Python ומציג פלט. היתרון הוא שניתן לשלב באותו מסמך הסבר, חישוב, טבלה וגרף. החיסרון הוא שתאים אינם תמיד רצים לפי סדר הופעתם: אם מריצים את תא 6 ואז חוזרים לתא 2, המשתנים בזיכרון עלולים להכיל ערכים שלא רואים כעת על המסך.

לכן הרגל מקצועי ראשון הוא הרצה מחדש מתחילת המחברת בסדר, ללא שימוש במשתנים שנוצרו במקרה בהרצה קודמת. בפרויקט גמר התלמיד צריך להוכיח שעמיתו או בוחן יכול לפתוח את המחברת ולשחזר את התוצאה.

דוגמה 1 — התא הראשון: שמירת נתונים והצגתם

```
# Notebook cell 1 – measurements
computer_name = "Dudu-PC-01"
temperature = 72.5
print(computer_name)
print(temperature)
```

בפקודה הראשונה הגדרנו משתנה שמחזיק טקסט. בשנייה הגדרנו משתנה שמחזיק מספר עשרוני. `print` אינו מאמן מודל ואינו משנה את הערכים: הוא רק מדפיס אותם כדי שנוכל לבדוק מה קרה.

לשאול לפני שמריצים איזה פלט יופיע? מה יקרה אם תא זה לא הורץ אבל תא אחר מנסה להשתמש ב־temperature? במקרה כזה תיתכן שגיאת NameError.

כיצד קוראים הודעת שגיאה?

כאשר שורה אינה פועלת, אין להעתיק מיד פתרון בלי להבין מה קרה. מתחילים מקריאת שורת השגיאה האחרונה: היא מתארת את סוג הבעיה. אחר כך מאתרים את השורה שאליה ההודעה מפנה ושואלים מה Python ציפה לקבל. שגיאה היא מידע על התנהגות התוכנית, לא רק כישלון של המתכנת.

קטע לדיון בלבד — מדוע סדר הרצה חשוב?

```
# This will fail if temperature was never defined:
# print(temperature + 5)
```

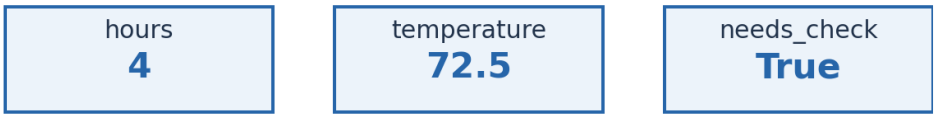
בדיקת הבנה

1. תארו שני מצבים שבהם מחברת נותנת פלט שונה בגלל סדר הרצת תאים.

2. כתבו תא שמגדיר שם מחשב, טמפרטורה ושעות שימוש; הדפיסו כל ערך בשורה משלו.

1.2 — משתנים, טיפוסים וחשבון מספרי

משתנה הוא שם שמאפשר לפנות לערך. ב־Python אין צורך להצהיר מראש על טיפוס המשתנה, אך לכל ערך יש טיפוס, והטיפוס קובע אילו פעולות אפשר לבצע עליו. מספר שלם (`int`) מתאים למשל למספר התקלות; מספר עשרוני (`float`) מתאים לטמפרטורה; מחרוזת (`str`) מתאימה לשם מחשב; ערך בוליאני (`bool`) מתאים לשאלה כן או לא.



Names refer to values; the value type matters

שלושה שמות משתנים והערכים שהם מפנים אליהם. הציור מדגיש שטיפוס הערך קובע את משמעותו.

טיפוס	דוגמה	משמעות בהקשר שלנו
int	faults = 3	מספר תקלות
float	temp = 72.5	טמפרטורה
str	name = "PC-01"	מזהה מחשב
bool	urgent = True	האם נדרש טיפול דחוף

דוגמה 2 — איך בודקים טיפוס?

```
faults = 3
temp = 72.5
name = "PC-01"
urgent = True

print(type(faults))
print(type(temp))
print(type(name))
print(type(urgent))
```

הסימן `=` מבצע השמה: לוקחים את התוצאה בצד ימין ומקשרים אותה לשם בצד שמאל. הוא אינו טענה מתמטית של שוויון. הביטוי `faults = faults + 1` תקין בתכנות: קודם מחשבים את מספר התקלות הקיים ועוד אחת, ואז מעדכנים את המשתנה.

דוגמה 3 — עדכון מצב

```
faults = 3
faults = faults + 1
print(faults)  # 4
```

אילו פעולות חשבון קיימות?

פעולה	דוגמה	תוצאה
+	7 + 2	9
-	7 - 2	5

פעולה	דוגמה	תוצאה
*	7 * 2	14
/	7 / 2	3.5
//	7 // 2	3
%	7 % 2	1
**	2 ** 3	8

שימו לב: `/` היא חלוקה רגילה ומחזירה מספר עשרוני גם כאשר החלוקה מדויקת, ואילו `//` היא חלוקת רצפה. עבור מספרים שליליים אין לפרש אותה תמיד כ״מחיקת החלק העשרוני״: למשל `-7 // 2` שווה -4. האופרטור `%` מספק שארית בהתאם לכללי Python ומשמש, בין היתר, לבדיקת זוגיות.

דוגמה פתורה: ממוצע ושליטה בסדר פעולות

דודו מדד שלוש טמפרטורות: 68, 82, 75. כדי לחשב ממוצע מחברים את הערכים ומחלקים במספרם. טעות נפוצה היא לכתוב `68 + 82 + 75 / 3`: לפי קדימות הפעולות רק המספר האחרון יחולק בשלוש.

$$\text{mean} = (68 + 82 + 75) / 3 = 75.0$$

דוגמה 4 — סוגריים מגנים על משמעות הנוסחה

```
t1, t2, t3 = 68, 82, 75
mean = (t1 + t2 + t3) / 3
print(mean)
```

למה זה חשוב ב־AI? אלגוריתם יוכל להשתמש בטמפרטורה ממוצעת בתור תכונה, אך תחילה עלינו לוודא שהחישוב מייצג את התכונה שהתכוונו אליה. מספר שהודפס ללא הודעת שגיאה עדיין עשוי להיות שגוי מבחינה מדעית.

תרגול מדורג

1. חשבו ביד ואז ב־Python את הממוצע של 60, 75, 90, 75.
2. הסבירו מה ההבדל בין `"5" + "2"` לבין `5 + 2`.
3. כתבו ביטוי המחזיר את מספר הדקות שנותרו לאחר חלוקת 137 דקות לשעות שלמות.
4. בדקו מה מחזיר `type(True)` ומדוע אין להשתמש ב־`True` כמדידה מספרית רגילה.

1.3 — מחרוזות, קלט והצגת תוצאות

הנתונים במערכת אינם רק מספרים. לפעמים צריך לייצג מזהה מחשב, שם כיתה, הודעת מצב או תאריך. כל אלה הם מחרוזות (`str`). מחרוזת היא רצף תווים: גם הספרות שבתוכה הן תווים ולא בהכרח מספרים שניתן לחבר ולממוצע. למשל `"72.5"` היא מחרוזת, בעוד `72.5` הוא מספר.

פונקציות כמו `float()` ו־`int()` מאפשרות להמיר טקסט לערכים מספריים כאשר הטקסט תקין. אם הוא מכיל מילה במקום מספר, ההמרה תיכשל; בהמשך נלמד כיצד לזהות ולטפל בכך.

דוגמה 5 — המרת טקסט ובדיקת תבנית

```
reading = "72.5"
value = float(reading)
print(value + 2)   # 74.5

computer = "PC-04"
print(computer.lower())
print(computer.startswith("PC-"))
```

פלט שקריא לאדם

הדפסת רשימת מספרים לבדה אינה מספרת לקורא מה המשמעות שלהם. לכן משתמשים במחרוזות מעוצבות (`f-string`), המאפשרות לשלב ערכים והסברים. הביטוי שבתוך הסוגריים המסולסלים מחושב לפני ההדפסה. ניתן לבקש שתי ספרות אחרי הנקודה כדי להציג ממוצע בצורה נוחה.

דוגמה 6 — תצוגה שאפשר להכניס לדוח

```
computer = "PC-04"
average = 75.333333
print(f"Computer: {computer}; average temperature: {average:.2f} C")
```

נבחין בין עיגול לצורך הצגה לבין שינוי הנתונים עצמם: הדפסת `75.33` אינה מוחקת את הדיוק של המשתנה `average`. חשוב לשמור את ערכי המקור, בייחוד לפני חישובי סטטיסטיקה או אימון מודל.

קלט מהמשתמש — מתי כן ומתי לא?

הפונקציה `input()` מאפשרת למשתמש להקליד ערך, אבל היא מחזירה מחרוזת. לכן לשם חישוב יש להמיר את הקלט. במחברות AI משתדלים לרוב להפריד בין קוד החישוב לבין קלט אינטראקטיבי, כדי שניתן יהיה להריץ את אותה מחברת באופן אוטומטי בלי להמתין למשתמש.

קוד לדיון — אינו מופעל אוטומטית במחברת

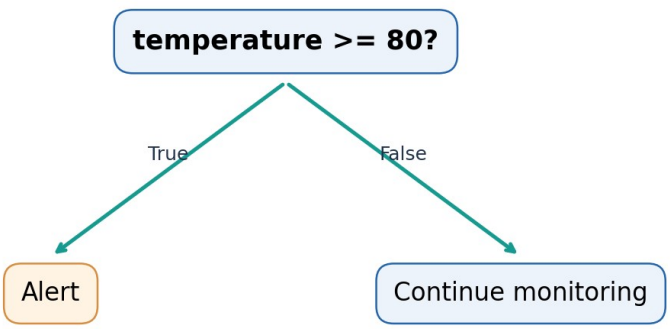
```
# Interactive example – run when you want to enter data:
# text = input("Temperature: ")
# temperature = float(text)
# print(temperature + 1)
```

בדיקת הבנה

1. כתבו שורת `f-string` המציגה מחשב וטמפרטורה עם ספרה אחת אחרי הנקודה.
2. הסבירו מדוע `int("4.5")` אינו מבצע אותה פעולה כמו `int(float("4.5"))`.
3. כתבו בדיקה שמזהה האם מזהה מחשב מתחיל באותיות `PC-`.

1.4 — תנאים ולוגיקה: מתיאור נתון להחלטה

מערכת לניטור מחשבים צריכה לעיתים לפעול אחרת לפי מצב המדידה. תנאי הוא ביטוי שמחזיר `True` או `False`. מבנה `if` מאפשר להריץ קוד רק כאשר התנאי מתקיים. החלק שמוזח פנימה שייך לבלוק התנאי; הזחה שגויה משנה את משמעות התוכנית או גורמת לשגיאה.



עץ החלטה פשוט: מדידה של 80 מעלות ומעלה מסומנת להתייחסות.

דוגמה 7 — החלטה בעזרת *if/else*

```
temperature = 82
if temperature >= 80:
    print("Temperature alert")
else:
    print("Temperature normal")
```

נוסחת ההחלטה אינה ״למידת מכונה״. היא כלל מפורש שנכתב על ידי המתכנת. ההבדל חשוב: בהמשך נלמד מודל שמנסה ללמוד כלל מנתוני עבר, אבל בשלב זה אנחנו עדיין כותבים את הכלל בעצמנו.

השוואות וצירופי תנאים

סימן	פירוש	דוגמה
==	שווה	temperature == 80
!=	שונה	faults != 0
>, >=	גדול / גדול או שווה	temperature >= 80
<, <=	קטן / קטן או שווה	faults <= 2
and	שני התנאים מתקיימים	temp > 80 and faults > 0
or	לפחות אחד מתקיים	temp > 90 or faults > 4
not	שלילת תנאי	not urgent

הביטוי `and` מחזיר אמת כאשר שני התנאים נכונים; `or` מחזיר אמת כאשר לפחות אחד נכון. בדוגמאות פשוטות הערכים הם בוליאניים, אך יש לזכור שב־Python אופרטורים אלה יכולים להחזיר גם אחד מהאופרנדים. כדי לא להקשות בתחילה נשתמש בביטויים שהשוואותיהם בוליאניות מפורשות.

דוגמה 8 — שלושה מצבים ולא רק שניים

```
temperature = 86
faults = 2
if temperature >= 80 and faults >= 2:
    status = "urgent"
elif temperature >= 80:
    status = "check temperature"
else:
    status = "normal"
print(status)
```

מקרה גבול דורש תשומת לב: האם 80 נחשב חריג או שהכלל חל רק מעל 80? תשובה כזו חייבת להופיע בהגדרת הבעיה. בקוד ההבדל הוא בין `>` לבין `>=`, אבל במערכת אמיתית מדובר בהחלטה עסקית או מדעית.

תרגול

1. עבור טמפרטורה 80, מה התוצאה של `temperature > 80` ומה של `temperature >= 80`?
2. כתבו תנאי שמזהה מחשב עם יותר משלוש תקלות או טמפרטורה של 90 ומעלה.
3. הסבירו מדוע כלל ידני בתנאי `if` אינו נחשב מודל שאומן על נתונים.

1.5 — לולאות, סכימה וספירה של נתונים

כאשר מתקבלות מאות מדידות, אי אפשר לכתוב פקודת חיבור נפרדת לכל מדידה. לולאה מאפשרת לבצע פעולה חוזרת לפי סדר על אוסף נתונים. נתחיל בלולאת `for` שמבקרת בכל מדידה, ולאחר מכן נכיר את `range` ואת לולאת `while` המתאימה למקרים שבהם ממשיכים עד שמתקיים תנאי עצירה.



Loop: visit each item and update a count or total

הלולאה עוברת לפי הסדר על ארבע מדידות; ערכים של 80 ומעלה מסומנים בצבע שונה.

דוגמה פתורה: סכום ומונה

דודו רוצה לחשב טמפרטורה ממוצעת וגם לדעת כמה מדידות הגיעו לסף 80. נצטרך שני משתנים: `total` יצבור את הסכום; `high_count` יספור מדידות חריגות. לפני תחילת הלולאה מאתחלים את שניהם באפס, כי עדיין לא עיבדנו אף מדידה.

דוגמה 9 — צבירה וספירה

```
temperatures = [68, 82, 76, 91]
total = 0
high_count = 0

for temperature in temperatures:
    total += temperature
    if temperature >= 80:
        high_count += 1

average = total / len(temperatures)
print(average)     # 79.25
print(high_count)  # 2
```

שלב	temperature	total לאחר עדכון	high_count
אתחול	—	0	0
1	6 8	6 8	0
2	8 2	150	1
3	76	226	1
4	9 1	3 1 7	2

טבלת המעקב חשובה יותר משינון התחביר: היא עוזרת להסביר מה נשאר נכון אחרי כל איטרציה. לאחר שני ערכים, למשל, `total` הוא סכום שני הערכים הראשונים ו־`high_count` הוא מספר הערכים שחצו את הסף מביניהם. הרעיון הזה יופיע בהמשך גם באלגוריתמים הלומדים מנתונים.

לולאה לפי אינדקס

דוגמה 10 — כאשר צריך גם את מיקום התצפית

```
for i in range(len(temperatures)):
    print(i, temperatures[i])
```

`range(len(...))` מחזיר אינדקסים מ־0 ועד אחד פחות ממספר האיברים. בהמשך, כאשר צריך לעבור על כמה מערכים מקבילים, חשוב לדעת מה מייצג האינדקס. אם לא צריך מיקום, נעדיף בדרך כלל מעבר ישיר על הערכים כדי לשמור על קוד פשוט.

מתי נשתמש ב־while?

דוגמה 11 — עם תנאי עצירה ברור *while*

```
index = 0
while index < len(temperatures):
    print(temperatures[index])
    index += 1
```

בלולאת `while` חובה לוודא שמשהו משתנה וכך מקדם אותנו לעבר תנאי העצירה. אם שכחנו `index += 1`, נקבל לולאה שאינה מסתיימת. ב־Colab אפשר לעצור את ההרצה, אבל עדיף ללמוד לזהות את הבעיה לפני שהיא מתרחשת.

זהירות: רשימה ריקה אם אין מדידות, `len(temperatures)` הוא אפס. חלוקה ב־0 תגרום ל־ZeroDivisionError. בהמשך נלמד טיפול בקלט חסר; כבר עכשיו חשוב להגדיר כיצד התוכנית צריכה להגיב.

תרגול העברה

1. חשבו ביד את הסכום ואת מספר המדידות מעל 85 ברשימה `[65, 86, 80, 92, 79]`.
2. כתבו לולאה המדפיסה רק מדידות בטווח 70–85 (כולל גבולות).
3. הסבירו מתי תעדיפו `for temperature in temperatures` ומתי תצטרכו `range(len(...))`.
4. תכננו שינוי שימנע חלוקה באפס במקרה של רשימה ריקה.

1.6 — רשימות, אינדקסים והעתקת נתונים

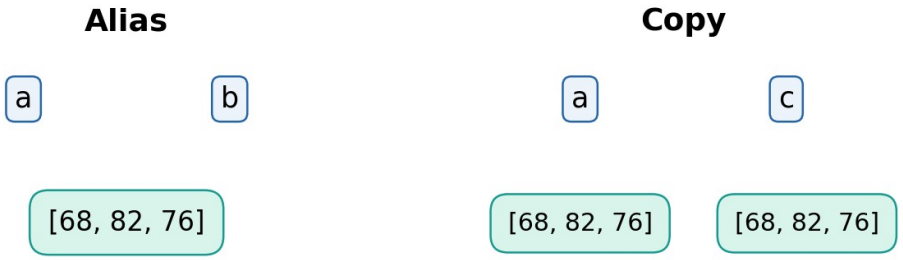
רשימה (`list`) מאפשרת לשמור כמה ערכים באוסף מסודר. האינדקסים מתחילים באפס, ולכן האיבר הראשון נמצא ב־`temperatures[0]`. רשימה ניתנת לשינוי: אפשר להוסיף איבר, להחליף איבר ולמחוק איבר. תכונה זו שימושית, אך גם דורשת זהירות כדי לא לשנות נתוני מקור בלי כוונה.

דוגמה 12 — גישה, חיתוך והוספה

```
temperatures = [68, 82, 76, 91]
print(temperatures[0])    # first value
print(temperatures[-1])   # last value
print(temperatures[1:3])  # values at indexes 1 and 2

temperatures.append(74)
print(len(temperatures))  # 5
```

בביטוי `[1:3]` הגבול השמאלי כלול והגבול הימני אינו כלול. לכן נקבל את האיברים במקומות 1 ו־2. חיתוך של רשימה יוצר רשימה חדשה; לעומת זאת, השמה `b = a` יוצרת שם נוסף לאותו אובייקט רשימה. ההבדל יכול ליצור טעויות שקשה לאתר בניסויי AI.



שני שמות יכולים להפנות לאותה רשימה; העתקה יוצרת רשימה עצמאית ברמה הראשונה.

דוגמה 13 — כינוי לעומת העתקה

```
original = [68, 82, 76]
alias = original
copy_of_values = original.copy()

alias[0] = 999
print(original)       # [999, 82, 76]
print(copy_of_values) # [68, 82, 76]
```

`copy()` יוצרת העתקה שטחית. עבור רשימה של מספרים זה מספיק בדרך כלל. אם הרשימה כוללת רשימות פנימיות, השינויים ברשימות הפנימיות עדיין עשויים להשפיע על שני המבנים. נרחיב על מבנים מקוננים רק לאחר שההבדל הבסיסי ברור.

משמעות ל־AI: לשמור נתוני מקור

נניח שתלמיד מחליף ברשימה את כל המדידות שמעל 80 בערך 80. זו פעולה שמתקנת לכאורה קלט חריג, אבל היא גם מוחקת מידע. אם לא נשמר עותק של נתוני המקור, קשה להבין מה בוצע וייתכן שנאמן בהמשך מודל על נתונים מעוותים. כל שלב בעיבוד נתונים צריך להיות מתועד ובעל הצדקה.

תרגול

1. הסבירו מה תחזיר הפקודה `temperatures[-2]` כאשר הרשימה כוללת ארבעה איברים.
2. כתבו קוד היוצר עותק של רשימת מדידות וממיר רק בעותק ערכים שליליים ל־0.
3. מדוע השורה `b = a` אינה מגינה על רשימת המקור?
4. מה יקרה אם ננסה לגשת ל־`temperatures[100]`? איך נמנעים מכך?

1.7 — מילונים, רשומות ומערכי נתונים פשוטים

כאשר לכל מחשב יש יותר ממאפיין אחד, לא נוח לשמור כמה רשימות מקבילות בלי הסבר. מילון (`dict`) שומר קשרים בין מפתחות לערכים, למשל `"name"` למזהה המחשב ו־`"temperature"` למדידה. רשימת מילונים מאפשרת לייצג אוסף רשומות, שכל אחת מהן מתארת תצפית אחת.

דוגמה 14 — תצפית כמילון

```
computer = {
    "name": "PC-01",
    "temperature": 72.5,
    "hours": 4,
    "faults": 1
}
print(computer["temperature"])
print(computer.get("location", "unknown"))
```

הגישה `computer["temperature"]` מניחה שהמפתח קיים; אם אינו קיים נקבל `KeyError`. לעומת זאת, `get` מאפשר להחזיר ערך חלופי כאשר המפתח חסר. אין לפרש ערך חסר כאפס באופן אוטומטי: אפס תקלות הוא נתון, ואילו ״אין לנו מידע על מספר תקלות״ הוא מצב אחר לגמרי.

דוגמה 15 — רשימה של תצפיות

```
computers = [
    {"name": "PC-01", "temperature": 72, "faults": 1},
    {"name": "PC-02", "temperature": 87, "faults": 3},
    {"name": "PC-03", "temperature": 76, "faults": 0}
]
```

```
for pc in computers:
    print(pc["name"], pc["temperature"])
```

מה ההבדל בין מפתח, מאפיין ותווית?

המפתח במילון הוא שם של שדה, למשל `temperature`. המאפיין (Feature) הוא ערך שמשתמשים בו כדי לתאר את התצפית בזמן החיזוי. התווית או משתנה המטרה (Target) היא התשובה שהמודל יתבקש לחזות, אם קיימת. בחירת תכונה אינה רק החלטה טכנית: אסור לכלול בין תכונות החיזוי מידע שלא יהיה זמין כאשר המערכת תידרש לפעול.

לדוגמה, אם רוצים לחזות האם מחשב יתקלקל מחר, מספר התקלות שיתרחשו בשבוע הבא אינו מאפיין לגיטימי של היום. בהמשך נקרא לבעיה של שימוש במידע עתידי בשם "זליגת נתונים" (Data Leakage).

היכרות שימושית — Set־ו Tuple

בנוסף לרשימה ולמילון יש שני מבנים נפוצים: `tuple` הוא רצף מסודר שאינו ניתן לשינוי לאחר יצירתו (אם כי אובייקטים משתנים בתוכו עשויים להשתנות), ו־`set` הוא אוסף ללא סדר אינדקסים קבוע וללא כפילויות. לא נעמיס כאן ניתוח מעמיק; התלמיד צריך לזהות אותם בקוד ולבחור מבנה נתונים מתאים.

דוגמה 16 — *tuple* לממדים ו־*set* לזיהוי ערכים ייחודיים

```
resolution = (8, 8)  # two dimensions
labels = ["ok", "alert", "ok", "check"]
unique_labels = set(labels)
print(resolution)
print(unique_labels)
```

תרגול

1. תכננו רשומת מחשב עם שדות `id`, `hours`, `temperature`, `status`.
2. הסבירו מדוע מידע חסר על תקלות אינו שקול לערך 0.
3. מה תהיה משמעות תווית `needs_repair` בפרויקט סיווג? מתי היא נודעת?
4. כתבו לולאה שמדפיסה את שמות המחשבים שהטמפרטורה שלהם 80 ומעלה.

1.8 — פונקציות: להפוך חישוב לפעולה ברורה ואמינה

תוכנית אמיתית מורכבת מכמה פעולות החוזרות במקומות שונים: בדיקת מדידות, חישוב ממוצע, יצירת סיכום, שמירת קובץ. אם נעתיק את אותן שורות שוב ושוב, כל תיקון יחייב אותנו למצוא את כל העותקים. פונקציה מאפשרת לתת לפעולה שם, להגדיר קלטים ולהחזיר תוצאה. זהו אחד המעברים החשובים מכתיבת פקודות בודדות לכתיבת מערכת שניתן לתחזק.



פונקציה מקבלת קלט ומחזירה תוצאה: אותו חישוב ניתן להפעלה על מדידות שונות.

דוגמה 17 — קלט, שם פעולה וערך מוחזר

```
def average_temperature(values):
    """Return the average of a non-empty sequence."""
    return sum(values) / len(values)

readings = [68, 82, 76]
result = average_temperature(readings)
print(result)
```

המילה `def` מגדירה פונקציה; `values` הוא פרמטר מקומי, שמקבל בזמן הקריאה את הרשימה שהעברנו; `return` מחזיר את התוצאה למקום שממנו קראנו לפונקציה. אם היינו כותבים `print` במקום `return`, הפונקציה הייתה מציגה ערך למסך אבל לא מאפשרת להשתמש בו בנוחות בחישובים הבאים.

הפרדה בין חישוב להצגה

דוגמה 18 — פונקציה בוליאנית וערך ברירת מחדל

```
def is_hot(temperature, threshold=80):
    return temperature >= threshold

print(is_hot(82))
print(is_hot(82, threshold=90))
```

הדוגמה גם מלמדת על ערך ברירת מחדל: אם המשתמש אינו מעביר סף, משתמשים ב־80; אם מעביר סף אחר, נקבל החלטה בהתאם אליו. חשוב לא לשנות ערכי סף סתם כדי להתאים תוצאה רצויה, אלא לקבוע אותם לפי הגדרת הבעיה ולתעד את ההחלטה.

בדיקות וקלט לא תקין

הפונקציה הקודמת לחלוקה בממוצע הניחה שהרשימה אינה ריקה. אפשר להפוך את ההנחה למפורשת ולשלוח הודעת שגיאה ברורה כאשר אין נתונים. לא תמיד צריך ״לתקן״ קלט לא תקין במקום; לעיתים נכון לעצור את החישוב כדי למנוע הפקת תוצאה מטעה.

דוגמה 19 — הגדרת תנאי תקינות לפונקציה

```
def average_temperature(values):
    if len(values) == 0:
        raise ValueError("Expected at least one temperature")
    return sum(values) / len(values)
```

משתנים שנוצרים בתוך הפונקציה אינם בהכרח זמינים מחוץ לה. זהו רעיון תחום ההכרה של משתנים (Scope): הוא מונע תלות מיותרת בין חלקי תוכנית. פרק התכנות מונחה העצמים יתבסס על היכולת הזאת להפריד אחריות בין רכיבים.

בדיקות עם assert

דוגמה 20 — שלוש בדיקות אוטומטיות קטנות

```
assert average_temperature([60, 80]) == 70
assert is_hot(80) is True
assert is_hot(79) is False
```

בדיקה טובה כוללת גם מקרה רגיל וגם מקרה גבול. עם זאת, `assert` איננו מנגנון אימות קלט למערכת ייצור: ניתן לבטל אותו בהרצות Python מסוימות. כאן הוא כלי לימודי מהיר לבדיקת נכונות של פונקציות. בהמשך נלמד לכתוב בדיקות מסודרות.

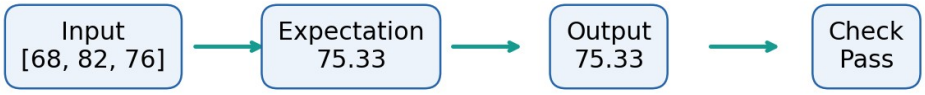
תרגול

1. כתבו פונקציה `count_hot(values, threshold=80)` המחזירה את מספר המדידות שעומדות בסף.
2. הסבירו מדוע פונקציה שמחזירה מספר עדיפה לעיתים על פונקציה שרק מדפיסה אותו.
3. הגדירו שלוש בדיקות: רשימה רגילה, רשימה שבה אף מדידה אינה חריגה ומקרה גבול בדיוק בסף.
4. תכננו מה הפונקציה אמורה לעשות כשהרשימה ריקה; נמקו את הבחירה.

1.9 — שגיאות, מודולים, קבצים ושחזור

תלמיד שמפתח מערכת AI חייב ללמוד לא רק לכתוב תוכנית שעובדת בדוגמה אחת, אלא גם לברר מדוע תוכנית אינה עובדת או מחזירה תוצאה חשודה. נהוג להבחין בין שגיאת תחביר (`SyntaxError`), שגיאת זמן ריצה כגון `IndexError` או `ValueError`, ושגיאת לוגיקה שבה הקוד רץ אך מחשב דבר אחר מהנדרש.

דרך בדיקה	דוגמה	סוג הבעיה
קריאת שורת השגיאה ותיקון הסימן	חסר : אחרי if	תחביר
בדיקת אורך הרשימה והאינדקס	גישה לאינדקס לא קיים	זמן ריצה
חישוב ידני והשוואה לפלט	ממוצע ללא סוגריים	לוגיקה
בירור מדיניות ערכים חסרים	טמפרטורה חסרה	נתונים



כל בדיקה משווה פלט שהתקבל לפלט צפוי, ולא רק בודקת שהקוד סיים לרוץ.

למה צריך import?

Python כוללת ספרייה תקנית עשירה, לצד ספריות חיצוניות. באמצעות `import` ניתן להשתמש בפונקציות שהוגדרו במודולים אחרים. `math` מספקת פעולות מתמטיות, `csv` קריאה של קבצי CSV, ו־`pathlib` טיפול בנתיבי קבצים. מודולים כגון `numpy` ו־`pandas` הם ספריות חיצוניות שמשמשות במיוחד במדעי הנתונים.

דוגמה 21 — מודולים מספרייה תקנית

```
from pathlib import Path
from math import sqrt

folder = Path(".")
print(folder.resolve())
print(sqrt(81))
```

קריאת CSV — לא להניח שכל קובץ זהה

קובץ CSV מייצג נתונים טבלאיים באמצעות טקסט. שורה אחת עשויה להכיל כותרות, והשורות הבאות ערכי תצפיות. אך אין להניח שהנתונים תקינים: ייתכנו שדות ריקים, מפרידים שונים ומספרים שנשמרו כטקסט. בפרק EDA נעמיק בבדיקות ובניקוי; כאן נלמד לזהות את המבנה ולקרוא קובץ בצורה בסיסית.

דוגמה 22 — יצירה וקריאה של *CSV* קטן

```
import csv
from pathlib import Path

path = Path("computers_demo.csv")
with path.open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "temperature", "hours"])
    writer.writerow(["PC-01", 72, 4])
    writer.writerow(["PC-02", 87, 8])

with path.open("r", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows)
```

הערכים ש־`csv.DictReader` קוראת הם בדרך כלל מחרוזות, ולכן לפני חישוב ממוצע יש לבצע המרות ולבדוק תקינות. זוהי דוגמה נוספת להבחנה בין סוג הנתון כפי שנשמר בקובץ לבין המשמעות המספרית שלו. בפרויקט גמר יש לשמור מקור וגרסת קובץ ולא לדרוס אותו ללא צורך.

בדיקה מול מקרה מוכר

דוגמה 23 — התאמה בין תוצאה ידנית למימוש

```
def calc_average(values):
    if not values:
        raise ValueError("No values")
    return sum(values) / len(values)

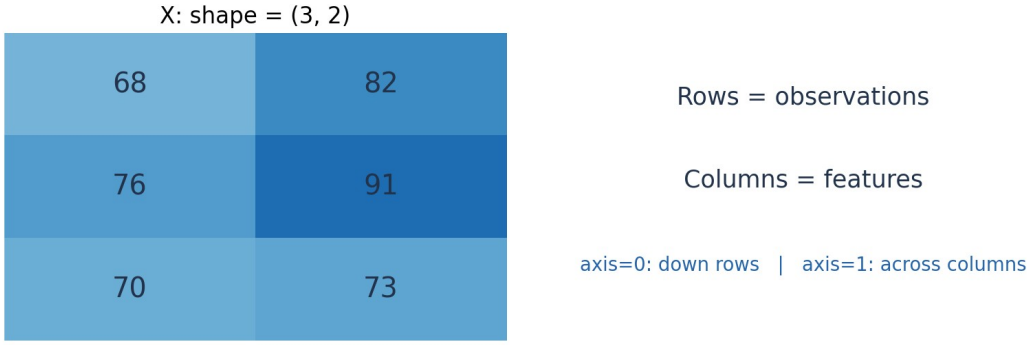
assert calc_average([68, 82, 76]) == 226 / 3
```

תרגול

1. תארו שגיאת לוגיקה שעלולה לעוות ממוצע גם כאשר אין הודעת שגיאה.
2. שנו את דוגמת ה־CSV כך שתכיל שלושה מחשבים ובדקו מה סוג הערכים שנקראו.
3. מדוע חשוב לרשום את מקור קובץ הנתונים ואת קוד ההכנה שלו בפרויקט AI?
4. תכננו בדיקה למקרה שבו קובץ אינו נמצא בתיקייה הרצויה.

1.10 — NumPy: מעבר מרשימות למערכים מספריים

רשימות Python נוחות לצעדים הראשונים, אך בעיבוד נתונים מספריים אנו עובדים לעיתים על אלפי ערכים, מטריצות ותמונות. NumPy מאפשרת לייצג מערכים רב־ממדיים בעלי טיפוס אחיד ולהפעיל עליהם פעולות וקטוריות. המשמעות היא לא רק כתיבה קצרה יותר: הצורה (`shape`) של המערך אומרת כמה תצפיות וכמה מאפיינים יש לנו.



מטריצה X בעלת שלוש שורות ושני מאפיינים. הצורה קובעת מה מייצג כל ציר.

דוגמה 24 — שלוש תצפיות ושני מאפיינים

```
import numpy as np

X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

print(X.shape)         # (3, 2)
print(X[:, 0])        # first feature for all computers
print(X.mean(axis=0)) # feature means
```

כאשר `axis=0` מפעילים חישוב לאורך שורות, כך שמתקבלת תוצאה לכל עמודה; ממוצע שתי העמודות בדוגמה הוא 75.33 לטמפרטורה ו־5.33 לשעות שימוש. כאשר `axis=1`, מתקבלת תוצאה לכל שורה. לא כדאי לחשב ממוצע של טמפרטורות ושעות יחד: היחידות שלהן שונות, והתוצאה אינה מדד בעל משמעות מוגדרת.

פעולות איבר־איבר ו־Broadcasting

דוגמה 25 — אותה פעולה על כל איבר

```
temperatures = np.array([68, 82, 76], dtype=float)
shifted = temperatures + 2
scaled = temperatures / 100
print(shifted)
print(scaled)
```

כאשר מוסיפים 2 למערך, NumPy מחילה את החיבור על כל איבר. לכן אפשר לחשוב על ההוראה כעל לולאה נסתרת, אך בביצוע המקצועי זו פעולה מערכית. Broadcasting מערכים בעלי צורות מתאימות יכולים להשתתף באותה פעולה בלי ליצור במפורש את כל העותקים. בפרקי הווקטורים ו־KNN שבספר הגדול נעמיק במנגנון ובתנאי ההתאמה בין ממדים.

דוגמה: הכנת מאפיינים לפני מודל

דוגמה 26 — מהו וקטור תכונות?

```
X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

feature_means = X.mean(axis=0)
print(feature_means)
print(X[0])
```

כל שורה ב־X מייצגת מחשב אחד וכל עמודה מאפיין. בעתיד נוסיף מערך `y` שבו נשמרות התוויות הידועות, אם מדובר בלמידה מונחית. חשוב לשמור תמיד על התאמה בין סדר השורות ב־X לבין סדר התוויות ב־y; ערבוב סדר אחד מהם ישבש את הלמידה גם אם הקוד רץ.

מיני־פרויקט מסכם: ניטור מעבדת המחשבים

דודו רוצה לקבל דוח קצר על מצב המעבדה. הוא צריך לוודא שיש נתונים, לחשב טמפרטורה ממוצעת, לספור מדידות שעברו סף, לכתוב פונקציה נפרדת לקבלת החלטה ולהכין מטריצת מאפיינים. זהו תהליך עיבוד נתונים מבוסס כללים; הוא עדיין אינו מודל למידת מכונה, משום שהמחשב לא למד את הכלל מתוך דוגמאות מסומנות.



Rule-based analysis is not yet a trained machine-learning model

תהליך מיני־הפרויקט: איסוף, בדיקה, סיכום, סימון והסבר התוצאה.

דוגמה 27 — מימוש מודרך של שלבי סיכום הנתונים

```
import numpy as np

readings = [68, 82, 76, 91]

def summarize(values, threshold=80):
    if len(values) == 0:
        raise ValueError("At least one reading is required")
    average = sum(values) / len(values)
    high_count = sum(value >= threshold for value in values)
    return average, high_count

average, high_count = summarize(readings)
print(f"Average: {average:.2f}; high readings: {high_count}")

X = np.array(readings, dtype=float).reshape(-1, 1)
print("Feature matrix shape:", X.shape)
```

`reshape(-1, 1)` משנה רשימה חד־ממדית למטריצה בעלת עמודה אחת. הסימון -1 מבקש מ־NumPy לחשב את מספר השורות הנדרש על פי כמות האיברים. עיבוד הנתונים כאן פשוט בכוונה, כדי שכל תלמיד יוכל להסביר אותו. בשלב מתקדם יותר נחליף את סף הכלל במודל שמאומן מנתונים, ונבדוק אותו על קלט שלא שימש לאימון.

משימת סיכום עצמאית — בלי קוד פתרון

במעבדת Colab המלווה את הפרק תבנו דוח למערכת ספרייה או למערכת ניטור אנרגיה על בסיס אותו רעיון, אך עם נתונים ושאלות אחרות. המחברת כוללת תאים להשלמת קוד, מקרים לבדיקה ושאלות לנימוק. אל תעתיקו את דוגמת דודו כפי שהיא: מטרת המשימה היא להוכיח שאתם יכולים להעביר את המושגים להקשר חדש.

שאלות חשיבה בסגנון בחינת ידע והגנה

1. דודו אומר ש־`X.shape == (4, 1)` פירושו ארבעה מאפיינים ותצפית אחת. האם הוא צודק? נמקו.

2. הסבירו כיצד ניתן לקבל תוכנית שרצה ללא שגיאות אך מפיקה תוצאה בלתי תקפה מבחינה מדעית.
3. מה ההבדל בין בדיקת כלל סף שנכתב ידנית לבין אימון מודל שלומד סף או גבול החלטה מנתונים?
4. כיצד הייתם מתעדים שינוי באופן חישוב הממוצע כדי לאפשר שחזור של ניסוי?

תכנית הוראה מוצעת ל־30 שעות

מפגש	מטרת ביצוע	תוצר אישי
1	Colab ומשתנים	מחברת נפתחת מתחילתה ומפיקה אותו פלט
2	טיפוסים וחישובים	ממוצע ויחידות מדידה מוסברים
3	מחרוזות וקלט	הצגת דוח קריא והמרת סוגי נתונים
4	תנאים	כלל החלטה ומקרי גבול
5	לולאות	סכימה וספירה עם טבלת מעקב
6	רשימות	גישה, חיתוך, העתקה ושמירת מקור
7	מילונים	רשומות בעלות שדות ברורים
8	פונקציות	פעולות עם return ובדיקות
9	שגיאות וקבצים	נקרא ונבדק CSV
10	NumPy ופרויקט	דוח מסכם ומטריצת מאפיינים

מילון מונחים מסכם

מונח	הסבר לתלמיד
Variable	שם שמתייחס לערך
Data type	הטיפוס שמגדיר אילו פעולות אפשריות על ערך
Condition	ביטוי שמוביל לענף פעולה
Loop	ביצוע חוזר של הוראות
List	אוסף מסודר וניתן לשינוי
Dictionary	אוסף של זוגות מפתח–ערך
Function	פעולה בעלת שם, פרמטרים ותוצאה אפשרית
Return	החזרת ערך אל קוד שקרא לפונקציה
Exception	אירוע שגיאה בזמן ריצה
Module	קובץ/יחידה המכילים קוד שניתן לייבא
NumPy array	מבנה מערך מספרי רב־ממדי
Shape	מספר האיברים לאורך כל ציר
Feature	מאפיין המשמש לתיאור תצפית
Target	משתנה המטרה שמנסים לחזות
Data leakage	שימוש במידע שאינו לגיטימי בזמן האימון או הבחינה

מקורות והקשר לספר הגדול

עוגן תכנית: מאגר Data-Science-ML-Course, מסלול ההתמחות, פרק 1 — היכרות חוזרת עם שפת תכנות (30 שעות). מקורות ההשראה להסברים הם ספרי יסוד מוכרים, אך הניסוח, הדוגמאות, האיורים וקוד התרגול בחוברת מקוריים.

Eric Matthes, Python Crash Course, 3rd ed., No Starch Press (2022) — https://nostarch.com/Python-crash-course-3rd-edition

Al Sweigart, Automate the Boring Stuff with Python, 3rd ed., No Starch Press (2025) — https://nostarch.com/automate-boring-stuff-python-3rd-edition

/Python Software Foundation, The Python Tutorial — https://docs.python.org/3/tutorial

מיפוי ההשלמות במסלול שלנו — https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/03-Deep-Learning-Python

לאחר חוברת זו נמשיך ל־פרק 2: תכנות מונחה עצמים (30 שעות), לפרק 3: מבוא למדעי הנתונים וללמידת מכונה (10 שעות) ולפרק 4: חקר נתונים — EDA (15 שעות). לאחר השלמת ארבעתם נחבר אותם לחוברת יסוד אחת ונמקם אותה בתחילת ספר הקורס המאוחד. אין כאן כפילות של לימוד האלגוריתמים והמתמטיקה שכבר פותחו בספר הגדול.

בדיקת עצמאות לפני המשך האם אתם מסוגלים להסביר מה מייצגת כל שורה בטבלת הנתונים, להראות איך חושב כל מספר, לאתר טעות בקוד, ולהריץ את המחברת מחדש ללא התערבות? אם כן — מוכנים לפרק התכנות מונחה העצמים.